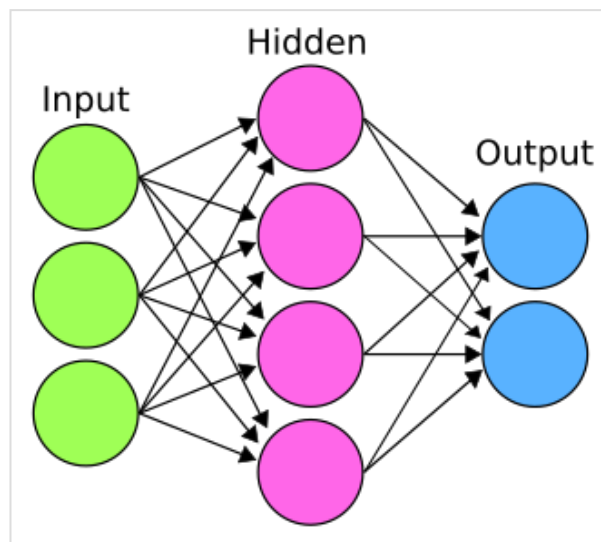




Mạng thần kinh nhân tạo

Não người hoạt động nhờ mạng lưới khoảng 86 tỷ tế bào thần kinh kết nối với nhau, truyền tín hiệu điện hóa để xử lý thông tin hay đưa ra quyết định. **Mạng thần kinh nhân tạo** (tiếng Anh: *Artificial neural network* hay chỉ đơn giản là *Neural network*, viết tắt ANN) là một mô hình học máy lấy cảm hứng từ cấu trúc đó: nó được cấu tạo thành từ các đơn vị xử lý đơn giản, kết nối với nhau theo lớp, cùng nhau học cách biểu diễn thông tin phức tạp từ dữ liệu.^[1] Nhờ vậy, mạng thần kinh nhân tạo có thể thực hiện các nhiệm vụ mà lập trình thủ công theo quy tắc gặp nhiều khó khăn, như nhận diện hình ảnh, dịch ngôn ngữ, hay tổng hợp giọng nói.

Về mặt kỹ thuật, một mạng thần kinh nhân tạo gồm các neuron nhân tạo (các nút) được tổ chức thành nhiều lớp: lớp đầu vào (*input layer*) nhận dữ liệu thô, một hoặc nhiều lớp ẩn (*hidden layer*) thực hiện các phép biến đổi phi tuyến, và lớp đầu ra (*output layer*) trả về kết quả.^[2] Mỗi kết nối giữa hai neuron mang một trọng số (*weight*), một con số xác định mức độ ảnh hưởng của tín hiệu đầu vào có thể được điều chỉnh trong quá trình huấn luyện. Đầu ra của mỗi neuron được tính bằng tổng có trọng số của các đầu vào, sau đó đưa qua một hàm kích hoạt (*activation function*) phi tuyến để tạo ra tín hiệu truyền sang lớp tiếp theo. Khi mạng có từ hai lớp ẩn trở lên, nó thường được gọi là mạng thần kinh sâu (*deep neural network*), là nền tảng của học sâu hiện đại.



Một mô hình mạng thần kinh nhân tạo đơn giản mô phỏng lại cách các tế bào thần kinh trong não người hoạt động. Lớp đầu tiên bên trái màu xanh lục là lớp đầu vào, lớp ở giữa màu tím là lớp ẩn, còn lớp bên phải màu xanh dương là lớp đầu ra. Mỗi mũi tên biểu thị kết nối từ đầu ra của một neuron đến neuron của lớp kế tiếp.

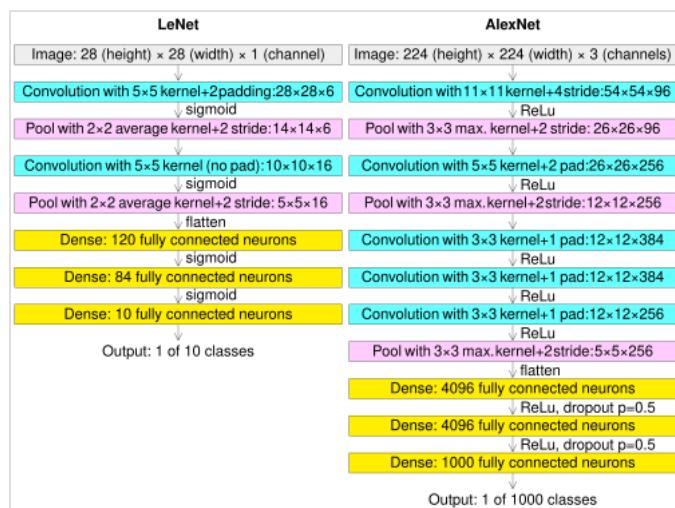
Lịch sử

Nền tảng lý thuyết và những bước đầu

Ý tưởng mô hình hóa tế bào thần kinh bằng toán học xuất hiện từ năm 1943, khi nhà thần kinh học Warren McCulloch và nhà logic học Walter Pitts đề xuất mô hình neuron nhị phân đầu tiên: một đơn vị tính toán nhận đầu vào nhị phân và cho ra đầu ra nhị phân dựa trên ngưỡng kích hoạt.^[3] Năm 1958, nhà tâm lý học Frank Rosenblatt hiện thực hóa ý tưởng này thành mô hình perceptron, một mạng thần kinh nhân tạo có thể huấn luyện được với quy tắc cập nhật trọng số tự động từ dữ liệu.^[4] Perceptron gây ra làn sóng lạc quan lớn và thu hút đầu tư mạnh từ chính phủ Mỹ. Tuy nhiên, năm 1969, Marvin Minsky và Seymour Papert chứng minh rằng perceptron đơn lớp không thể giải quyết các bài toán phi tuyến cơ bản như hàm XOR, khiến nghiên cứu về mạng thần kinh nhân tạo rơi vào giai đoạn trì trệ kéo dài.^[5]

Mạng thần kinh nhân tạo được hồi sinh vào thập niên 1980 với sự phổ biến hóa của thuật toán lan truyền ngược (*backpropagation*) bởi David Rumelhart, Geoffrey Hinton và Ronald Williams vào năm 1986.^[6] Thuật toán này cho phép huấn luyện hiệu quả các mạng nhiều lớp bằng cách tính gradient của hàm mất mát ngược từ lớp đầu ra về lớp đầu vào, giải quyết được bài toán XOR và mở ra khả năng xây dựng các mạng sâu hơn như mạng perceptron nhiều lớp (*multi-layer perceptron*, MLP). Đây là tiền đề kỹ thuật trực tiếp cho toàn bộ học sâu hiện đại.

Mạng tích chập và mạng hồi quy



Sơ đồ kiến trúc của hai mạng tích chập cho xử lý hình ảnh gây tiếng vang lớn đầu tiên: LeNet (trái) và AlexNet (phải).

Năm 1989, Yann LeCun và cộng sự áp dụng lan truyền ngược để huấn luyện mạng tích chập (*convolutional neural network*, CNN) nhận diện chữ số viết tay trên bưu kiện.^[7] Kiến trúc LeNet-5 hoàn thiện năm 1998 là mạng tích chập đầu tiên được triển khai thực tế ở quy mô lớn – các ngân hàng Mỹ dùng nó để tự động đọc số tài khoản trên séc. CNN khai thác cấu trúc không gian của hình ảnh bằng cách dùng các bộ lọc (*filter*) trượt qua ảnh thay vì kết nối đầy đủ từng điểm ảnh, giảm đáng kể số tham số và giúp mô hình không bị ảnh hưởng bởi thay đổi vị trí.

Song song với CNN, một nhánh khác của mạng thần kinh nhân tạo phát triển để xử lý dữ liệu tuần tự như văn bản và giọng nói: mạng hồi quy (*recurrent neural network*, RNN). Năm 1990,

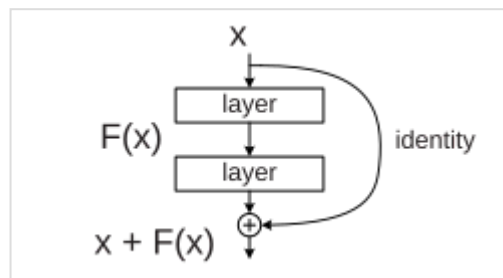
Jeffrey Elman giới thiệu kiến trúc RNN nền tảng – còn gọi là Elman network – trong đó lớp ẩn có kết nối vòng lặp trở lại chính nó: đầu ra của lớp ẩn ở bước thời gian trước được đưa trở lại làm một phần đầu vào cho bước tiếp theo, tạo ra một dạng "bộ nhớ" về ngữ cảnh trước đó.^[8] Tuy nhiên, RNN chuẩn gặp vấn đề triệt tiêu gradient (*vanishing gradient*) khi chuỗi đầu vào dài khiến gradient nhỏ dần qua mỗi bước thời gian khi lan truyền ngược, khiến mô hình gần như không ghi nhớ được các ngữ cảnh quá xa về trước. Năm 1997, Sepp Hochreiter và Jürgen Schmidhuber giới thiệu kiến trúc Long Short-Term Memory (LSTM), giải quyết vấn đề này bằng cơ chế cổng (*gate*) kiểm soát luồng thông tin được giữ lại hay loại bỏ qua từng bước thời gian.^[9] LSTM trở thành kiến trúc nền tảng cho nhận diện giọng nói và dịch máy trong suốt hai thập kỷ sau. Năm 2014, Kyunghyun Cho và cộng sự đề xuất Gated Recurrent Unit (GRU), đơn giản hóa cơ chế cổng của LSTM từ ba xuống còn hai cổng, nhẹ hơn để huấn luyện và trong nhiều tác vụ cho kết quả tương đương.^[10]

Kỷ nguyên học sâu: ImageNet và các cuộc thi ILSVRC

Bước ngoặt của học sâu hiện đại gắn liền với cuộc thi ImageNet Large Scale Visual Recognition Challenge (ILSVRC) – một benchmark thường niên dùng bộ dữ liệu hơn 14 triệu ảnh có nhãn của Stanford.^[11] Năm 2012, mô hình AlexNet của Alex Krizhevsky, Ilya Sutskever và Geoffrey Hinton giành chiến thắng với tỉ lệ lỗi top-5 là 15,3%, thấp hơn đối thủ hạng hai gần 11 điểm phần trăm, một khoảng cách chưa từng thấy.^[12] AlexNet về cơ bản là LeNet được mở rộng lớn hơn nhiều, huấn luyện trên GPU và sử dụng hàm kích hoạt ReLU thay vì sigmoid, một hàm đơn giản hơn nhưng hội tụ nhanh hơn và tránh được triệt tiêu gradient. Chiến thắng này khiến toàn bộ cộng đồng nghiên cứu chuyển hướng sang CNN.

Năm 2014, nhóm của Karen Simonyan và Andrew Zisserman tại Oxford giới thiệu VGGNet, đạt tỉ lệ lỗi 7,3% và về nhì tại ILSVRC. Những điểm đáng chú ý không phải là thứ hạng mà là câu hỏi kiến trúc mà kiến trúc này trả lời: mạng càng nhiều lớp thì hiệu suất có tăng không?^[13] VGGNet với 16-19 lớp, chỉ dùng các bộ lọc 3×3 nhỏ xếp chồng lên nhau, cho thấy câu trả lời dường như là có, khi thiết kế đơn giản, nhất quán của nó trở thành tham chiếu kiến trúc cho nhiều năm sau. Còn người chiến thắng năm ấy với tỉ lệ lỗi 6,7%, GoogLeNet (Inception v1) của Google đi theo hướng khác: thay vì tăng chiều sâu, nó giới thiệu module Inception cho phép mỗi lớp học song song ở nhiều kích thước bộ lọc khác nhau, giữ số tham số ở mức thấp hơn VGG nhiều lần do không sử dụng quá nhiều lớp.^[14]

Năm 2015, ResNet của Kaiming He và cộng sự tại Microsoft Research giành chiến thắng với tỉ lệ lỗi 3,57%, lần đầu tiên vượt qua ngưỡng lỗi của con người (~5%) trên tập dữ liệu này.^[15] ResNet giải quyết bài toán cốt lõi cản trở việc xây dựng mạng rất sâu: khi số lớp tăng quá nhiều, độ chính xác huấn luyện bắt đầu giảm, không phải do quá khớp mà do gradient dần biến mất qua quá nhiều lớp tương tự như vấn đề xảy ra với RNN. Đây là lý do không phải cứ thêm thật nhiều lớp ắt cho VGGNet là có thể giải quyết mọi bài toán. Giải pháp mà ResNet đưa ra là kết nối tắt (*residual connection*): tín hiệu đầu vào của một khối được cộng trực tiếp vào đầu ra của khối đó, tạo ra "đường tắt" cho gradient đi qua. ResNet-152 có tới 152 lớp nhưng vẫn huấn luyện ổn định, một kỳ tích tại thời điểm đó.



"Đường tắt" của ResNet. Kể cả khi đầu ra tính toán bằng 0 do gradient bị triệt tiêu, giá trị đầu vào ban đầu vẫn được truyền lên phía trên.

Transformer

Năm 2017, nhóm nghiên cứu tại Google công bố kiến trúc Transformer trong bài báo "Attention Is All You Need", thay thế hoàn toàn cơ chế hồi quy tuần tự bằng cơ chế chú ý (*attention*).^[16] Thay vì xử lý chuỗi từng từ một như RNN, Transformer tính toán mối quan hệ giữa mọi cặp vị trí trong chuỗi cùng lúc, cho phép song song hóa hoàn toàn và nắm bắt các kiến thức phụ thuộc từ nhiều bước trước mà không bị triệt tiêu gradient. Transformer nhanh chóng thay thế LSTM trong xử lý ngôn ngữ tự nhiên, và là nền tảng cho toàn bộ các mô hình ngôn ngữ lớn từ GPT đến Gemini.^{[17][18]} Năm 2020, kiến trúc Vision Transformer (ViT) mở rộng cơ chế attention sang thị giác máy tính bằng cách chia ảnh thành các mảnh nhỏ và xử lý như một chuỗi, cho thấy Transformer có thể cạnh tranh và vượt qua CNN trên nhiều tác vụ nhận diện hình ảnh.^[19]

Huấn luyện

Hàm mất mát

Để huấn luyện một mạng thần kinh nhân tạo, cần có một thước đo định lượng mức độ sai lệch giữa đầu ra của mô hình và kết quả mong muốn thông qua hàm mất mát.

Với bài toán hồi quy, hàm mất mát phổ biến nhất là sai số toàn phương trung bình:

$$L = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

trong đó y_i là nhãn thực tế và $\hat{y}_i$ là giá trị mô hình dự đoán cho mẫu thứ i .

Với bài toán phân loại, hàm mất mát thường dùng là cross-entropy:

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

Mục tiêu của huấn luyện là tìm bộ trọng số của toàn bộ mạng w sao cho hàm mất mát đạt giá trị nhỏ nhất trên tập dữ liệu huấn luyện.

Lan truyền ngược

Để tối thiểu hóa hàm mất mát, cần tính gradient của nó theo từng tham số trong mạng, tức là hàm mất mát thay đổi bao nhiêu khi mỗi trọng số thay đổi một lượng nhỏ. Lan truyền ngược là thuật toán tính gradient này một cách hiệu quả bằng cách áp dụng quy tắc dây chuyền (*chain rule*) của giải tích, lần lượt từ lớp đầu ra ngược về lớp đầu vào.^[6] Với một lớp bất kỳ, gradient của hàm mất mát theo trọng số w được tính theo công thức:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w}$$

trong đó mỗi nhân tử được tính từ lớp liền sau rồi nhân dồn ngược lại qua các lớp trước. Nhờ đó, mạng có thể xác định chính xác mỗi trọng số đóng góp bao nhiêu vào sai số đầu ra, và điều chỉnh tương ứng.

Thuật toán suy giảm độ dốc

Sau khi có gradient, thuật toán suy giảm độ dốc được sử dụng để cập nhật trọng số theo hướng ngược chiều gradient để giảm hàm mất mát:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

trong đó η là tốc độ học (*learning rate*), kiểm soát sự thay đổi trọng số nhiều hay ít mỗi lần cập nhật. Trong thực tế, thay vì tính gradient trên toàn bộ tập dữ liệu, người ta thường dùng *stochastic gradient descent* (SGD) để tính gradient trên từng nhóm dữ liệu nhỏ, vừa nhanh hơn vừa giúp mô hình thoát khỏi các điểm cực tiểu địa phương. Một biến thể phổ biến hơn là Adam (*Adaptive Moment Estimation*), kết hợp hai cơ chế: tích lũy động lượng (*momentum*) từ các gradient trước để tăng tốc ở hướng ổn định, và điều chỉnh tốc độ học riêng cho từng tham số dựa trên lịch sử gradient của nó, trong đó $\hat{m}$ là ước lượng moment bậc một (trung bình động của gradient) và $\hat{v}$ là ước lượng moment bậc hai (trung bình động của bình phương gradient):^[20]

$$w \leftarrow w - \frac{\eta}{\sqrt{\hat{v}} + \epsilon} \hat{m}$$

Hay nói cách khác: nếu tham số nào đang được cập nhật nhanh sau nhiều bước liên tục thì càng thay đổi nhiều hơn, còn tham số nào đang thay đổi ít thì được cập nhật bớt lại. Tương tự như việc đẩy một viên đá xuống đồi, nếu đồi dốc khiến viên đá lăn xuống nhanh thì nó sẽ tiếp tục di chuyển về hướng đó, còn nếu

sườn dốc thoải thì nó sẽ từ từ lăn chậm lại. Adam hiện là optimizer mặc định trong hầu hết các framework học sâu nhờ hội tụ nhanh và ít nhạy cảm với việc chọn learning rate.

Tính phổ quát

Tính phổ quát (generalization) là khả năng của mô hình hoạt động tốt trên dữ liệu chưa từng thấy trong quá trình huấn luyện. Đây là mục tiêu trung tâm của mọi hệ thống học máy, bởi một mô hình chỉ hoạt động tốt trên dữ liệu huấn luyện nhưng thất bại với dữ liệu mới thực tế thì không có giá trị ứng dụng. Hiện tượng khi mô hình "học vẹt" dữ liệu huấn luyện thay vì học quy luật tổng quát được gọi là quá khớp (*overfitting*). Các kỹ thuật phổ biến để kiểm soát quá khớp bao gồm chính quy hóa (*regularization*), "bỏ học" một vài neuron (*dropout*), tăng cường dữ liệu (*data augmentation*), hoặc tham chiếu với một tập dữ liệu không được sử dụng trực tiếp trong quá trình học.^[21] Ở chiều ngược lại, thiếu khớp (*underfitting*) xảy ra khi mô hình quá đơn giản để nắm bắt được quy luật trong dữ liệu.

Tính chất lý thuyết

Định lý xấp xỉ toàn năng

Định lý xấp xỉ toàn năng (*universal approximation theorem*) là nền tảng lý thuyết giải thích tại sao mạng thần kinh nhân tạo có khả năng học được hầu hết các hàm số. Phiên bản gốc, được chứng minh độc lập bởi George Cybenko (1989) và Kurt Hornik (1991), phát biểu rằng một mạng thần kinh chỉ với một lớp ẩn duy nhất nhưng đủ rộng (có nhiều neuron) có thể xấp xỉ bất kỳ hàm liên tục nào trên một miền compact với độ chính xác tùy ý.^{[22][23]} Nói cách khác, về mặt lý thuyết, mạng nông đã đủ mạnh để biểu diễn mọi hàm số, song định lý này không nói gì về số lượng neuron cần thiết cụ thể, và trong thực tế số đó có thể lớn đến mức không thể huấn luyện được.

Điều này dẫn đến phiên bản thứ hai của định lý, phát triển trong thập niên 2010: thay vì mạng rộng, mạng đủ sâu (tức là đủ nhiều lớp) cũng có khả năng xấp xỉ toàn năng, và thường làm được với số tham số ít hơn đáng kể so với mạng nông tương đương.^[24] Kết quả này cung cấp cơ sở lý thuyết cho xu hướng xây dựng mạng ngày càng sâu hơn trong học sâu hiện đại, do chiều sâu không chỉ là lựa chọn tốt về mặt thực nghiệm (như đã thấy với VGGNet) mà còn mang lại hiệu quả biểu diễn về mặt toán học.

Định lý không có bữa trưa miễn phí

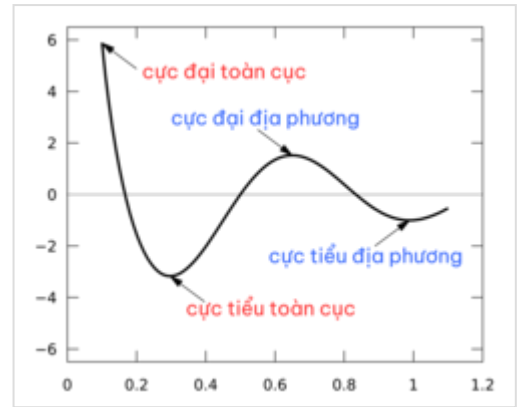
Định lý không có bữa trưa miễn phí (*no free lunch theorem*), được chứng minh bởi David Wolpert và William Macready năm 1997, phát biểu rằng không tồn tại một thuật toán (hay kiến trúc) học máy nào là "tốt nhất", là tốt hơn mọi thuật toán khác trên tất cả các bài toán, có nghĩa là khi lấy trung bình trên toàn bộ các phân phối dữ liệu có thể có, mọi thuật toán đều có hiệu suất như nhau.^[25] CNN vượt trội trên ảnh vì khai thác cấu trúc không gian cục bộ, Transformer hiệu quả với chuỗi dài vì cơ chế attention toàn cục, nhưng không kiến trúc nào tối ưu trên tất cả. Kể cả khi Vision Transformer có hiệu quả ngang ngửa với CNN trên ImageNet,^[19] nó vẫn cần một lượng lớn dữ liệu để học trong khi các mô hình CNN hoàn toàn có thể "chịu đói" dữ liệu mà vẫn có thể học được các đặc trưng trên dữ liệu.^[26] Lựa chọn kiến trúc, hàm mất mát, và phương pháp huấn luyện phù hợp với đặc điểm cụ thể của dữ liệu và bài toán là điều kiện tiên quyết để mạng thần kinh hoạt động tốt.

Hội tụ

Một câu hỏi lý thuyết quan trọng là liệu quá trình huấn luyện có đảm bảo tìm được bộ tham số tối ưu không.

Câu trả lời ngắn gọn là **không**.

Điều này là do gradient descent chỉ đảm bảo hội tụ về một điểm cực tiểu địa phương, không nhất thiết là cực tiểu toàn cục. Không gian tham số của mạng sâu là một hàm nhiều chiều cực kỳ phức tạp, chứa vô số điểm cực tiểu địa phương và điểm yên ngựa (*saddle point*). Tuy nhiên, nghiên cứu thực nghiệm và lý thuyết gần đây cho thấy với mạng đủ lớn, hầu hết các cực tiểu địa phương đều có giá trị hàm mất mát gần với cực tiểu toàn cục,^[27] và các kỹ thuật như SGD với mini-batch và momentum giúp mô hình thoát khỏi các điểm yên ngựa hiệu quả hơn. Ngoài ra, khi chiều rộng của mạng tiến đến vô cực, hành vi huấn luyện có thể được mô tả xấp xỉ bằng một mô hình tuyến tính thông qua *Neural Tangent Kernel*, cho phép phân tích hội tụ chặt chẽ hơn về mặt toán học.^[28]



Hàm mất mát có thể có các điểm cực tiểu địa phương khiến thuật toán suy giảm độ dốc bị "mắc kẹt" bên trong, trong khi mục tiêu là tìm được điểm cực tiểu toàn cục.

Tham khảo

1. "Artificial Neural Network - an overview | ScienceDirect Topics". www.sciencedirect.com. Truy cập ngày 8 tháng 4 năm 2026.
2. Bishop, Christopher M. (2006). *Pattern Recognition and Machine Learning*. Springer. ISBN 978-0-387-31073-2.
3. McCulloch, Warren S.; Pitts, Walter (1943). "A logical calculus of the ideas immanent in nervous activity". *The bulletin of mathematical biophysics*. Quyển 5 số 4. tr. 115–133. doi:10.1007/BF02478259.
4. Rosenblatt, F. (1958). "The perceptron: A probabilistic model for information storage and organization in the brain". *Psychological Review*. Quyển 65 số 6. tr. 386–408. doi:10.1037/h0042519.
5. Minsky, Marvin; Papert, Seymour (1969). *Perceptrons: An Introduction to Computational Geometry*. MIT Press. doi:10.7551/mitpress/11301.001.0001. ISBN 978-0-262-34393-0.
6. Rumelhart, David E.; Hinton, Geoffrey E.; Williams, Ronald J. (1986). "Learning representations by back-propagating errors". *Nature*. Quyển 323 số 6088. tr. 533–536. doi:10.1038/323533a0.
7. Lecun, Y.; Bottou, L.; Bengio, Y.; Haffner, P. (1998). "Gradient-based learning applied to document recognition". *Proceedings of the IEEE*. Quyển 86 số 11. tr. 2278–2324. doi:10.1109/5.726791.
8. Elman, Jeffrey L. (1990). "Finding structure in time". *Cognitive Science*. Quyển 14 số 2. tr. 179–211. doi:10.1207/s15516709cog1402_1.
9. Hochreiter, Sepp; Schmidhuber, Jürgen (1997). "Long Short-Term Memory". *Neural Computation*. Quyển 9 số 8. tr. 1735–1780. doi:10.1162/neco.1997.9.8.1735.
10. Cho, Kyunghyun; van Merriënboer, Bart; Gulcehre, Caglar; Bahdanau, Dzmitry; Bougares, Fethi; Schwenk, Holger; Bengio, Yoshua (2014). "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation". *arXiv*. arXiv:1406.1078. doi:10.48550/arXiv.1406.1078.

11. Deng, Jia; Dong, Wei; Socher, Richard; Li, Li-Jia; Kai Li; Li Fei-Fei (2009). "ImageNet: A large-scale hierarchical image database". *CVPR 2009*. tr. 248–255. doi:10.1109/CVPR.2009.5206848.
12. Krizhevsky, Alex; Sutskever, Ilya; Hinton, Geoffrey E. (2012). "ImageNet Classification with Deep Convolutional Neural Networks". *Advances in Neural Information Processing Systems*. Quyển 25.
13. Simonyan, Karen; Zisserman, Andrew (2014). "Very Deep Convolutional Networks for Large-Scale Image Recognition". *arXiv*. arXiv:1409.1556.
14. Szegedy, Christian; Liu, Wei; Jia, Yangqing; Sermanet, Pierre; Reed, Scott; Anguelov, Dragomir; Erhan, Dumitru; Vanhoucke, Vincent; Rabinovich, Andrew (2015). "Going Deeper With Convolutions". tr. 1–9.
15. He, Kaiming; Zhang, Xiangyu; Ren, Shaoqing; Sun, Jian (2016). "Deep Residual Learning for Image Recognition". *CVPR 2016*. tr. 770–778. doi:10.1109/CVPR.2016.90.
16. Vaswani, Ashish; Shazeer, Noam; Parmar, Niki; Uszkoreit, Jakob; Jones, Llion; Gomez, Aidan N; Kaiser, Łukasz; Polosukhin, Illia (2017). "Attention is All you Need". *Advances in Neural Information Processing Systems*. Quyển 30. Curran Associates, Inc.
17. Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). Improving language understanding by generative pre-training.
18. Team, G., Anil, R., Borgeaud, S., Alayrac, J. B., Yu, J., Soricut, R., ... & Blanco, L. (2023). Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.
19. Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., ... & Houlsby, N. (2020). An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*.
20. Kingma, Diederik P.; Ba, Jimmy (2014). "Adam: A Method for Stochastic Optimization". *arXiv*. arXiv:1412.6980. doi:10.48550/arXiv.1412.6980.
21. Lin, David Chuan-En (ngày 7 tháng 6 năm 2020). "8 Simple Techniques to Prevent Overfitting".
22. Cybenko, George (1989). "Approximation by superpositions of a sigmoidal function". *Mathematics of Control, Signals and Systems*. Quyển 2 số 4. tr. 303–314. doi:10.1007/BF02551274.
23. Hornik, Kurt (1991). "Approximation capabilities of multilayer feedforward networks". *Neural Networks*. Quyển 4 số 2. tr. 251–257. doi:10.1016/0893-6080(91)90009-T.
24. Lu, Zhou; Pu, Hongming; Wang, Feicheng; Hu, Zhiqiang; Wang, Liwei (2017). "The Expressive Power of Neural Networks: A View from the Width". *Advances in Neural Information Processing Systems*. Quyển 30.
25. Wolpert, David H.; Macready, William G. (1997). "No free lunch theorems for optimization". *IEEE Transactions on Evolutionary Computation*. Quyển 1 số 1. tr. 67–82. doi:10.1109/4235.585893.
26. Steiner, Andreas; Kolesnikov, Alexander; Zhai, Xiaohua; Wightman, Ross; Uszkoreit, Jakob; Beyer, Lucas (ngày 18 tháng 9 năm 2024). "How to train your ViT? Data, Augmentation, and Regularization in Vision Transformers". *Transactions on Machine Learning Research*. arXiv:2106.10270.
27. Choromanska, Anna; Henaff, Mikael; Mathieu, Michael; Arous, Gerard Ben; LeCun, Yann (ngày 21 tháng 2 năm 2015). "The Loss Surfaces of Multilayer Networks". *Proceedings of the Eighteenth International Conference on Artificial Intelligence and Statistics* (bằng tiếng Anh). PMLR. tr. 192–204.
28. Jacot, Arthur; Gabriel, Franck; Hongler, Clément (ngày 3 tháng 12 năm 2018). "Neural tangent kernel: convergence and generalization in neural networks". *Proceedings of the 32nd International Conference on Neural Information Processing Systems*. Red Hook, NY, USA: Curran Associates Inc. tr. 8580–8589.

Liên kết ngoài

- Usenet FAQ: comp.ai.neural-nets (<http://www.faqs.org/faqs/ai-faq/neural-nets/>)
- Connectionism at MindDict (<http://artsci.wustl.edu/~philos/MindDict/connectionism.html>) Lưu trữ (<https://web.archive.org/web/20061014041142/http://artsci.wustl.edu/~philos/MindDict/connectionism.html>) ngày 14 tháng 10 năm 2006 tại [Wayback Machine](#)
- Chương trình mạng nơ ron đa lớp (<http://spiceneuro.wordpress.com/vietnamese/>) (Multi Layer Neural Network) và mạng nơ ron tự tổ chức (Self Organizing Maps) có giải thích bằng tiếng Việt.
- Sử dụng phần mềm mạng nơ ron 3 lớp Spice-MLP (<http://spiceneuro.files.wordpress.com/2013/07/spice-neuro-guide-vietnamese1.pdf>)
- Sử dụng phần mềm mạng tự tổ chức Spice-SOM (<http://spiceneuro.files.wordpress.com/2013/07/spice-som-guide-vietnamese1.pdf>)
- Hướng dẫn sử dụng mạng nơ ron trong các ứng dụng thực tế (http://spiceneuro.files.wordpress.com/2013/07/neural_network_practical_use_vi3.pdf) trong đó có minh họa phân loại ảnh khuôn mặt, ảnh người đi bộ, ảnh xe hơi, dự báo chứng khoán và một số ví dụ khác

Lấy từ "https://vi.wikipedia.org/w/index.php?title=Mạng_thần_kinh_nhân_tạo&oldid=75470455"